Conduite de projet

Décodage d'erreurs quantiques par réseau de neurones sur graphes

Version de travail – octobre 2026

Table des matières

1 Résumé exécutif et objectif du projet **3**

2 0. Comprendre le problème physique et mathématique **3**
2.1 Pourquoi faut-il corriger des erreurs quantiques ? . . . 3
2.2 Qu'est-ce qu'une erreur physique ? . . . 3
2.3 Pourquoi ne pas mesurer directement si une erreur a eu lieu ? . . . 4
2.4 Qu'est-ce qu'un syndrome ? Qu'est-ce qu'un détecteur ? . . . 4
2.5 Exemple simple : pourquoi les détecteurs sont-ils utiles ? . . . 5
2.6 Pourquoi répéter les mesures dans le temps ? . . . 5
2.7 Qu'est-ce qu'une erreur logique ? . . . 5
2.8 Qu'est-ce que le code de surface et sa distance ? . . . 5
2.9 Comment le décodeur agit-il en pratique ? . . . 6
2.10 Les trois niveaux à ne pas confondre . . . 6

3 1. Problématique, hypothèses et périmètre **6**
3.1 Problématique principale . . . 6
3.2 Hypothèses expérimentales . . . 7
3.3 Périmètre recommandé . . . 7

4 2. Méthode et protocole expérimental **7**
4.1 Étape A : générer un circuit bruité avec Stim . . . 7
4.2 Étape B : produire les exemples étiquetés . . . 8
4.3 Étape C : construire la référence PyMatching . . . 8
4.4 Étape D : calculer le taux d'erreur logique . . . 8
4.5 Étape E : construire les graphes pour le GNN . . . 9
4.6 Étape F : architecture initiale du GNN . . . 9
4.7 Étape G : séparation des données . . . 10
4.8 Étape H : généralisation au changement de bruit . . . 10

5 3. Métriques et analyse statistique **10**
5.1 Métrique principale : taux d'erreur logique . . . 10
5.2 Différence entre les décodeurs . . . 10
5.3 Incertitude statistique . . . 11
5.4 Temps de calcul . . . 11
5.5 Tableau de résultats à remplir . . . 11

6 4. État de l'art et références commentées **11**
6.1 Lange et al. : le travail directement étudié 11
6.2 Higgott : PyMatching et l'appariement de poids minimal 12
6.3 Google Quantum AI : le contexte expérimental 13
6.4 Mihailov et al. : une autre approche neuronale 14
6.5 Stim : outil de simulation et convention des détecteurs 14
6.6 Synthèse comparative 14

7 5. Que peut apporter ce projet au-delà d'une reproduction ? **14**
7.1 Réponse honnête 14
7.2 Extension recommandée : généralisation au taux de bruit 15
7.3 Extension alternative : ablation de la représentation du graphe 15
7.4 Extension expérimentale Google : intérêt et limites 15
7.5 Quel apport peut-on revendiquer ? 16

8 6. Planning opérationnel de 20 heures **16**
8.1 Critères de réussite 16
8.2 Risques et réponses prévues 17

9 7. Reproductibilité et bonnes pratiques **17**

10 8. Références bibliographiques **18**

1 Résumé exécutif et objectif du projet

Ce projet étudie un problème concret de correction d'erreurs quantiques : à partir des résultats de mesures de contrôle effectuées pendant une mémoire quantique bruitée, déterminer si l'information logique a subi une erreur. Le projet compare deux méthodes de décodage :

— **PyMatching**, un décodeur classique par appariement de poids minimal (*minimum-weight perfect matching*, MWPM), qui exploite un modèle de bruit fourni ;

— **un réseau de neurones sur graphes** (*graph neural network*, GNN), entraîné sur des exemples simulés étiquetés.

Question scientifique principale. Pour des codes de surface de distances $d = 3$ et $d = 5$, sous un modèle de bruit dépolarisant au niveau du circuit, quelle différence de taux d'erreur logique observe-t-on entre un GNN entraîné sur des exemples simulés et PyMatching, lorsque les deux méthodes sont évaluées sur les mêmes tirages de test indépendants ? Quel est le coût de calcul associé à chaque méthode ?

Extension ciblée. On évaluera aussi, si le temps le permet, la généralisation du GNN à un taux de bruit de test différent de celui de l'entraînement. Cette expérience permet de mesurer une forme précise de robustesse au décalage de distribution ; elle ne prouve pas à elle seule une robustesse à tous les modèles de bruit.

Portée honnête. Le projet est d'abord une reproduction contrôlée et une étude de sensibilité, pas l'annonce d'un nouveau décodeur de l'état de l'art. Son intérêt dépendra de la rigueur du protocole, de la qualité des mesures et de l'analyse des conditions dans lesquelles le GNN se rapproche – ou non – de PyMatching.

2 0. Comprendre le problème physique et mathématique

2.1 Pourquoi faut-il corriger des erreurs quantiques ?

Un qubit est un système quantique dont l'état peut s'écrire

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle, \qquad |\alpha|^2 + |\beta|^2 = 1.$$

Les amplitudes α et β sont des nombres complexes. Une mesure dans la base computationnelle donne 0 ou 1 avec des probabilités $|\alpha|^2$ et $|\beta|^2$. Contrairement à un bit classique, l'information quantique ne peut pas être copiée arbitrairement (théorème de non-clonage), et une mesure directe du qubit logique risque de détruire l'information qu'on souhaite préserver.

Les qubits physiques subissent des perturbations : erreurs de relaxation, déphasage, erreurs de portes, erreurs de préparation et erreurs de mesure. On utilise donc un *code correcteur quantique* : l'information logique est encodée collectivement dans plusieurs qubits physiques. Les mesures de contrôle détectent des incompatibilités révélatrices d'erreurs sans mesurer directement l'état logique.

2.2 Qu'est-ce qu'une erreur physique ?

Dans le formalisme de Pauli, les erreurs élémentaires usuelles sur un qubit sont :

$$X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, \quad Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}, \quad Y = iXZ = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix}.$$

— X échange $|0\rangle$ et $|1\rangle$: on l'interprète comme une erreur de bit-flip.

— Z change le signe relatif entre les amplitudes : c'est une erreur de phase-flip.

— Y combine les deux effets, à une phase globale près.

Une erreur physique peut aussi être une erreur de mesure ou de porte dans le circuit qui extrait le syndrome. Dans une simulation au niveau du circuit, il ne faut donc pas imaginer que les seules erreurs se produisent sur les qubits de données : des fautes peuvent survenir pendant les portes de préparation, les portes d'extraction du syndrome, les opérations de remise à zéro et les mesures.

Modèle dépolarisant. À titre d'exemple, un canal dépolarisant à un qubit peut être défini par

$$\mathcal{D}_p(\rho) = (1-p)\rho + \frac{p}{3}(X\rho X + Y\rho Y + Z\rho Z),$$

où ρ est la matrice densité de l'état et p la probabilité qu'une erreur de Pauli non triviale se produise. Les conventions exactes varient selon l'emplacement de l'erreur et le simulateur. Le paramètre `after_clifford_depolarization` de Stim ne signifie pas que toutes les sources de bruit du circuit sont nécessairement réglées au même taux. Le protocole doit donc déclarer explicitement chaque paramètre de bruit.

2.3 Pourquoi ne pas mesurer directement si une erreur a eu lieu ?

La mesure de l'état logique détruirait généralement l'information que la mémoire doit préserver. Le code mesure à la place des observables de contrôle appelées *stabilisateurs*. Un stabilisateur est un opérateur de Pauli S_i tel que les états du sous-espace code satisfont

$$S_i|\psi_{\text{code}}\rangle = +|\psi_{\text{code}}\rangle.$$

On mesure plusieurs stabilisateurs qui commutent entre eux. Chaque résultat est $+1$ ou -1 (ou, de façon équivalente, un bit 0 ou 1 après convention de codage). Ces mesures donnent des informations sur les erreurs sans révéler directement l'état logique encodé.

2.4 Qu'est-ce qu'un syndrome ? Qu'est-ce qu'un détecteur ?

Il faut distinguer trois objets souvent confondus.

1. Résultat de mesure. Notons $m_{i,t} \in \{0,1\}$ le résultat binaire de la mesure du stabilisateur numéro i au cycle t, après conversion du résultat physique en bit.

2. Événement de détection. Un détecteur compare des résultats de mesure qui devraient être compatibles en l'absence d'erreur. Dans le cas courant de mesures répétées d'un même stabilisateur, on peut construire

$$d_{i,t} = m_{i,t} \oplus m_{i,t-1}.$$

Ici $\oplus$ désigne le XOR, ou addition modulo 2 : $0 \oplus 0 = 0$, $0 \oplus 1 = 1$, $1 \oplus 0 = 1$ et $1 \oplus 1 = 0$. Si le résultat ne change pas, $d_{i,t} = 0$; s'il change, $d_{i,t} = 1$. Un détecteur vaut donc 1 lorsqu'une relation attendue entre mesures est violée. Pour le premier cycle, la relation peut comparer le résultat à une préparation connue ; au dernier cycle, des relations particulières peuvent impliquer la mesure finale. La formule ci-dessus est une intuition utile, mais la définition exacte des détecteurs dépend du circuit et de ses annotations.

3. Syndrome. Le syndrome d'un tirage est le vecteur de tous les événements de détection :

$$\mathbf{s} = (d_1, d_2, \ldots, d_{N_D}) \in \{0,1\}^{N_D},$$

où N_D est le nombre de détecteurs annotés dans le circuit. Le syndrome n'est pas nécessairement un unique bit : il contient les résultats de tous les détecteurs, souvent répartis dans l'espace et au cours du temps.

Un détecteur qui vaut 1 est souvent décrit comme « activé » ou « allumé ». Cela ne veut pas dire qu'il identifie exactement le qubit fautif. Plusieurs erreurs physiques différentes peuvent activer les mêmes détecteurs ; certaines erreurs peuvent même ne pas produire de changement de syndrome mais agir sur l'information logique. Le décodeur doit inférer l'effet logique à partir de l'ensemble des détecteurs.

2.5 Exemple simple : pourquoi les détecteurs sont-ils utiles ?

Considérons un stabilisateur idéal mesuré trois fois. Sans erreur, supposons que les résultats soient

$$(m_1, m_2, m_3) = (0, 0, 0).$$

Les deux détecteurs temporels valent alors

$$(d_2, d_3) = (m_2 \oplus m_1, m_3 \oplus m_2) = (0, 0).$$

Si une erreur de mesure fait basculer uniquement la mesure centrale, on observe par exemple $(0, 1, 0)$ et donc

$$(d_2, d_3) = (1, 1).$$

Deux détecteurs s'activent : l'incohérence apparaît à l'entrée et à la sortie temporelles de l'erreur de mesure. En revanche, une erreur persistante sur un qubit de données peut produire un motif différent, souvent spatialement localisé. C'est pourquoi les détecteurs sont naturellement organisés dans l'espace et le temps.

Cet exemple est schématique. Dans un vrai circuit quantique, les détecteurs sont définis par les relations de parité exactes entre mesures, y compris les effets des portes d'extraction du syndrome et des conditions aux limites.

2.6 Pourquoi répéter les mesures dans le temps ?

Une mesure de stabilisateur peut elle-même être fautive. Une seule série de mesures ne permet donc pas toujours de distinguer une erreur de mesure d'une erreur sur les qubits de données. On répète les mesures sur plusieurs cycles. La structure spatio-temporelle du syndrome aide alors à différencier les motifs d'erreur. Pour une mémoire de code de surface de distance d, on choisit souvent un nombre de cycles proportionnel à d pour un protocole de mémoire standard, mais la valeur exacte dépend de l'expérience simulée.

2.7 Qu'est-ce qu'une erreur logique ?

Un code protège un qubit logique encodé dans plusieurs qubits physiques. Le décodeur ne cherche pas nécessairement à identifier exactement chaque erreur physique survenue. Il doit choisir une correction qui ramène l'état dans le sous-espace code et préserve l'information logique.

Deux suites d'erreurs physiques peuvent produire le même syndrome, mais avoir des effets logiques différents. Leur différence peut correspondre à un opérateur logique non trivial. Une correction peut donc annuler les défauts locaux tout en appliquant, sans le vouloir, une opération logique X_L ou Z_L sur le qubit encodé. C'est ce qu'on appelle un échec logique.

Dans Stim, l'observable logique est suivi au moyen d'une annotation telle que `OBSERVABLE_`$INCLUDE.Pourun$
$lebitobservablederéférenceestlerésultatlogiqueeffectivementobservédansletirage, etledécodeurpréditleretour$

2.8 Qu'est-ce que le code de surface et sa distance ?

Le code de surface est un code correcteur quantique défini sur un réseau bidimensionnel, avec des stabilisateurs locaux de type X et Z. Les interactions locales permettent une extraction du syndrome compatible avec des architectures matérielles de qubits voisins.

La *distance* d du code est la longueur minimale, en poids de Pauli, d'un opérateur logique non trivial. Pour un code de surface standard, augmenter d augmente la taille du code et le nombre d'erreurs physiques nécessaires pour former une chaîne logique non triviale. Cela améliore la protection lorsque le bruit se situe dans le régime approprié, mais augmente aussi le coût matériel et computationnel. Les distances $d = 3$ et $d = 5$ sont de petits cas adaptés à un prototype ; ils ne suffisent pas à établir des conclusions d'échelle pour de grandes mémoires quantiques.

2.9 Comment le décodeur agit-il en pratique ?

Le circuit bruité est simulé ; chaque tirage (*shot*) produit :

1. un vecteur de détecteurs activés s ;
2. une ou plusieurs valeurs de référence pour les observables logiques ;
3. éventuellement, des informations supplémentaires pour analyser le tirage, mais qui ne doivent pas être données au décodeur si elles ne sont pas disponibles en expérience réelle.

Le décodeur reçoit le syndrome s et prédit le retournement de l'observable logique. On compare ensuite la prédiction à l'observable de référence produite par le simulateur. Le décodeur n'a pas accès à la description complète des fautes qui ont réellement eu lieu : ce serait une fuite d'information et rendrait l'évaluation artificiellement facile.

2.10 Les trois niveaux à ne pas confondre

Niveau	Question
Erreur physique	Quelle faute de porte, de préparation, de mesure ou de qubit s'est produite ?
Événements de détection	Quelles relations attendues entre les mesures sont violées ?
Erreur logique	Après la correction choisie, l'information logique est-elle incorrecte ?

Le décodeur résout le troisième problème en exploitant le deuxième ; il ne reconstruit pas nécessairement le premier.

3 1. Problématique, hypothèses et périmètre

3.1 Problématique principale

Pour $d \in \{3, 5\}$ et pour plusieurs taux physiques p, on compare le taux d'erreur logique d'un GNN entraîné sur des exemples simulés à celui de PyMatching, sur un ensemble de test commun et indépendant. On mesure aussi le temps d'inférence et le temps d'entraîneme

Quelle différence de qualité et de coût observe-t-on entre le GNN et PyMatching dans un prot

La question n'est pas « le GNN est-il meilleur en général ? » mais « dans quelles conditions précises, sur quelle tâche et avec quelle incertitude, le GNN atteint-il tel niveau de performance ? »

3.2 Hypothèses expérimentales

H1 – apprentissage. Le GNN peut apprendre une relation utile entre le syndrome et l'observable logique à partir d'exemples simulés.

H2 – performance. Le taux d'erreur logique du GNN peut être proche de celui de PyMatching, mais ce résultat n'est pas garanti et doit être mesuré.

H3 – coût. L'entraînement du GNN nécessite un coût initial, tandis que le temps d'inférence peut être compétitif. Cette hypothèse dépend de l'architecture, du matériel et de la taille des lots.

H4 – généralisation. Un GNN entraîné à un taux de bruit donné peut voir ses performances varier lorsque le taux de bruit change. On mesurera cette variation sans supposer qu'elle sera nécessairement défavorable.

3.3 Périmètre recommandé

- Code de surface en mémoire, avec une tâche Stim clairement identifiée.
- Distances $d = 3$ et $d = 5$.
- Bruit dépolarisant au niveau du circuit, avec tous les paramètres déclarés explicitement.
- PyMatching comme référence principale.
- Un GNN de petite taille, puis éventuellement une ablation simple.
- Données simulées pour l'expérience principale ; données Google uniquement en extension après validation du pipeline.

Il faut fixer la tâche exacte avant de commencer : par exemple surface_code:rotated_memory ou surface_code:rotated_memory_x. Les deux tâches ne doivent pas être mélangées sans explication, car elles protègent des observables logiques différentes.

4 2. Méthode et protocole expérimental

4.1 Étape A : générer un circuit bruité avec Stim

Stim est un simulateur optimisé pour les circuits de correction d'erreurs, notamment les circuits de Clifford. Un exemple minimal est :

Listing 1 – Exemple de circuit de mémoire de code de surface

```
import stim

d = 3
p = 0.005

circuit = stim.Circuit.generated(
    "surface_code:rotated_memory_z",
    distance=d,
    rounds=d,
    after_clifford_depolarization=p,
    before_round_data_depolarization=p,
    before_measure_flip_probability=p,
    after_reset_flip_probability=p,
)
```

Ce code est un point de départ et non une définition universelle du « bruit dépolarisant ». Il règle ici plusieurs sources de bruit à la même valeur pour créer un scénario explicite. Pour l'expérience finale, documenter chaque paramètre et vérifier les hypothèses du modèle.

4.2 Étape B : produire les exemples étiquetés

On échantillonne les détecteurs et les observables logiques séparément :

Listing 2 – Échantillonnage des syndromes et observables

```
import numpy as np

sampler = circuit.compile_detector_sampler()
syndrome, observables = sampler.sample(
    shots=10000,
    separate_observables=True,
)

print(syndrome.shape)     # (nombre de tirs, nombre de d tecteurs)
print(observables.shape)  # (nombre de tirs, nombre d'observables)
```

La matrice syndrome contient les entrées accessibles au décodeur. La matrice observables contient les cibles logiques de référence. Elles doivent rester séparées : donner les observables au GNN en entrée constituerait une fuite de cible.

4.3 Étape C : construire la référence PyMatching

Listing 3 – Décodeur MWPM de référence

```
import pymatching

dem = circuit.detector_error_model(decompose_errors=True)
matching = pymatching.Matching.from_detector_error_model(dem)

prediction_pm = matching.decode_batch(syndrome)
```

Le modèle d'erreur de détecteurs (*detector error model*, DEM) décrit les mécanismes de fautes sous la forme de relations entre détecteurs et observables. L'option decompose_erro demande une décomposition des mécanismes pour obtenir une représentation compatible avec le graphe de PyMatching dans les cas pris en charge. Il faut vérifier les avertissements de Stim : une décomposition n'est pas une garantie que toute corrélation complexe est parfaitement représentée par un graphe simple.

4.4 Étape D : calculer le taux d'erreur logique

Pour une observable logique, la métrique est :

$$\widehat{p_{\mathrm{L}}} = \frac{1}{N} \sum_{i=1}^{N} \mathbf{1}[\hat{o}_i \neq o_i] \,,$$

où N est le nombre de tirages, $o_i \in \{0, 1\}$ l'observable de référence, $\hat{o}_i$ la prédiction du décodeur et $\mathbf{1}[\cdot]$ vaut 1 si la condition est vraie et 0 sinon.

Listing 4 – Calcul de l'erreur logique

```
num_failures = np.count_nonzero(prediction_pm != observables)
logical_error_rate = num_failures / observables.size
print("Taux d'erreur logique :", logical_error_rate)
```

Pour une seule observable, cette formule est simple. Si plusieurs observables sont prédites, il faut définir la métrique avant l'expérience. Pour le taux de tirages contenant au moins une erreur logique :

$$\widehat{p_{\mathrm{L_{any}}}} = \frac{1}{N} \sum_{i=1}^{N} \mathbf{1}[\exists j : \hat{o}_{ij} \neq o_{ij}] \,.$$

Ne pas confondre cette quantité avec le taux d'erreurs calculé par bit d'observable. Pour le premier projet, une tâche à une observable logique évite cette ambiguïté.

4.5 Étape E : construire les graphes pour le GNN

Un graphe est un objet mathématique $G = (V, E)$: V est l'ensemble des nœuds et E l'ensemble des arêtes reliant certains nœuds. Dans un GNN, chaque nœud porte des attributs numériques et les couches du réseau échangent des informations le long des arêtes.

Choix de représentation. Une première idée consiste à créer un nœud uniquement pour chaque détecteur activé. Cette représentation est compacte, mais elle peut donner un graphe vide et perdre la distinction entre absence d'un événement et absence d'un nœud. Une solution plus stable pour un premier prototype est de conserver tous les détecteurs potentiels comme nœuds et de leur attribuer un bit d'activation $d_i \in \{0, 1\}$. On peut ajouter les coordonnées spatio-temporelles (x_i, y_i, t_i) lorsque le circuit les fournit.

Les arêtes ne doivent pas être choisies uniquement parce que deux points sont proches en distance euclidienne. Elles devraient refléter une structure pertinente du code ou des relations d'erreur. Le modèle de détecteurs de Stim peut servir de référence pour comprendre quelles combinaisons de détecteurs sont reliées par des mécanismes de faute. Une construction par k plus proches voisins reste possible comme variante, mais elle doit être motivée et évaluée comme un choix de modèle.

Attention à la cible. Le GNN doit prédire l'observable logique de référence à partir du syndrome, pas la localisation complète de l'erreur physique. En général, plusieurs histoires d'erreurs peuvent être compatibles avec un même syndrome. La cible est donc une classe logique, et non une vérité unique sur les fautes microscopiques.

4.6 Étape F : architecture initiale du GNN

Une couche de passage de messages peut s'écrire de façon générique :

$$\mathbf{h}_i^{(\ell+1)} = \phi\Big(\mathbf{h}_i^{(\ell)}, \mathrm{AGG}_{j\in\mathcal{N}(i)}\, \psi(\mathbf{h}_i^{(\ell)}, \mathbf{h}_j^{(\ell)}, \mathbf{e}_{ij})\Big),$$

où :

- $\mathbf{h}_i^{(\ell)}$ est le vecteur de caractéristiques du nœud i à la couche ℓ ;
- $\mathcal{N}(i)$ est l'ensemble de ses voisins ;
- $\mathbf{e}_{ij}$ représente éventuellement les attributs de l'arête ;
- ψ calcule un message à partir du nœud et de son voisin ;
- AGG agrège les messages (somme, moyenne ou maximum) ;
- ϕ met à jour la représentation du nœud.

Après plusieurs couches, une opération de lecture globale rassemble les représentations en un vecteur de graphe, par exemple

$$\mathbf{h}_G = \frac{1}{|V|}\sum_{i\in V} \mathbf{h}_i^{(L)}.$$

Une tête de classification prédit ensuite une probabilité de retournement logique :

$$\hat{q} = \sigma(f_\theta(\mathbf{h}_G)), \qquad \sigma(z) = \frac{1}{1+e^{-z}}.$$

Le paramètre θ représente les poids appris du réseau. Le modèle peut être entraîné avec la perte d'entropie croisée binaire :

$$\mathcal{L}(\theta) = -\frac{1}{N}\sum_{i=1}^{N}\left[y_i \log \hat{q}_i + (1-y_i)\log(1-\hat{q}_i)\right],$$

où y_i est l'étiquette binaire et $\hat{q}_i$ la probabilité prédite. En pratique, BCEWithLogitsLoss est généralement préférable à l'application manuelle du logarithme et de la sigmoïde, pour des raisons numériques.

Ces équations décrivent une famille d'architectures. Pour prétendre reproduire Lange et al., il faudra reprendre leur définition effective du graphe, des caractéristiques, de la cible et de la procédure d'entraînement plutôt que supposer que ce GNN générique est identique au leur.

4.7 Étape G : séparation des données

Séparer les tirages avant l'entraînement :

- entraînement : ajuste les paramètres du GNN ;
- validation : choisit les hyperparamètres et l'arrêt de l'entraînement ;
- test : fournit le résultat final, consulté après les décisions de modèle.

Les tirages étant indépendants lorsqu'ils sont échantillonnés séparément du même circuit, le risque principal de fuite est surtout une mauvaise organisation du code ou l'utilisation accidentelle des étiquettes de test pour choisir le modèle. Les mêmes échantillons de test doivent servir à PyMatching et au GNN, afin que la comparaison soit appariée.

4.8 Étape H : généralisation au changement de bruit

On entraîne par exemple à $p_{\text{train}} = 0.005$, puis on teste à $p_{\text{test}} \in \{0.0025, 0.005, 0.01\}$, sous réserve que ces valeurs soient adaptées au protocole choisi. Pour chaque valeur de test, on génère un circuit correspondant. PyMatching doit être construit à partir du modèle de bruit de ce circuit de test, tandis que le GNN est conservé sans réentraînement pour mesurer son transfert.

Il s'agit d'une comparaison volontairement asymétrique : PyMatching reçoit le modèle de bruit correct au test, le GNN non. C'est un test de robustesse d'un décodeur appris, pas une comparaison où les deux méthodes disposent des mêmes informations a priori.

5 3. Métriques et analyse statistique

5.1 Métrique principale : taux d'erreur logique

La métrique centrale est le taux d'erreur logique p_{L}. Il mesure la fraction des tirages pour lesquels le décodeur ne prédit pas correctement l'observable logique. Une accuracy élevée peut être trompeuse lorsque les échecs sont rares ; il faut donc rapporter directement le taux d'erreur logique et le nombre absolu d'échecs.

5.2 Différence entre les décodeurs

Pour un même point expérimental, on rapporte la différence absolue

$$\Delta p_{\text{L}} = p_{\text{L}}^{\text{GNN}} - p_{\text{L}}^{\text{PM}}$$

et, lorsque p_{L}^{PM} n'est pas trop proche de zéro, l'écart relatif

$$\delta_{\text{rel}} = \frac{p_{\text{L}}^{\text{GNN}} - p_{\text{L}}^{\text{PM}}}{p_{\text{L}}^{\text{PM}}}.$$

Un écart relatif positif signifie que le GNN échoue plus souvent ; un écart négatif signifie qu'il échoue moins souvent. Si le dénominateur est nul ou très faible, l'écart relatif est instable : présenter alors la différence absolue et les intervalles de confiance.

5.3 Incertitude statistique

Si K échecs sont observés sur N tirages indépendants, l'estimateur est $\hat{p} = K/N$. Pour une première analyse, un intervalle de confiance binomial à 95 % (par exemple Wilson ou Clopper-Pearson) est préférable à une approximation gaussienne lorsque K est petit.

Si aucun échec n'est observé, cela ne prouve pas que le vrai taux est nul. Une règle approximative courante donne une borne supérieure à 95 % d'environ $3/N$ dans ce cas (« règle de trois »). Ainsi, zéro échec sur 1 000 tirs indique seulement que le taux est compatible avec une valeur de l'ordre de quelques 10^{-3} ou moins ; il ne permet pas de mesurer fiablement un taux de 10^{-5}.

Un objectif pratique est de viser environ 100 échecs observés par point de mesure lorsque le coût de simulation le permet. Le nombre de tirs nécessaire dépend du taux réel : si le taux est très faible, il faudra beaucoup plus de tirs. Il est donc utile de commencer par une courte simulation pilote.

5.4 Temps de calcul

Mesurer séparément :

1. temps de génération des échantillons ;
2. temps de construction du DEM et de PyMatching ;
3. temps d'entraînement du GNN ;
4. temps d'inférence, après échauffement éventuel, sur un nombre de tirages annoncé.

Ne pas mélanger le temps de génération du circuit au temps de décodage. Préciser le processeur, le GPU éventuel, les tailles de lots et le nombre de répétitions du benchmark. Le débit peut être exprimé en tirs par seconde. Une comparaison du temps d'inférence doit être interprétée avec prudence : PyMatching et un GNN peuvent ne pas utiliser le même matériel ni la même stratégie de traitement par lots.

5.5 Tableau de résultats à remplir

d	p	N	Échecs GNN	Échecs PyMatching	Temps GNN / PM
3					
3					
5					
5					

Pour chaque ligne, ajouter dans le rapport les taux calculés, leurs intervalles de confiance, les graines utilisées et la configuration exacte du bruit.

6 4. État de l'art et références commentées

6.1 Lange et al. : le travail directement étudié

Référence. M. Lange et al., *Data-driven decoding of quantum error correcting codes using graph neural networks*, *Physical Review Research*, 7, 023181 (2025), version arXiv:2307.0 version révisée en mars 2025. DOI : https://doi.org/10.1103/PhysRevResearch.7.023181.

Problème posé. Les décodeurs optimaux au sens de la vraisemblance peuvent être coûteux. Les décodeurs efficaces, tels que le matching, utilisent une structure algorithmique et un modèle d'erreur. Leur qualité peut dépendre de la justesse de ce modèle et de la capacité de la représentation à capturer les corrélations. Lange et ses coauteurs étudient une autre voie : apprendre directement, à partir d'exemples, à associer une configuration de détecteurs à une classe d'erreur logique.

Formulation. Le syndrome est transformé en un graphe annoté. Le GNN agrège l'information locale et produit une prédiction de la classe logique. Le modèle apprend à partir de données simulées ; les observables logiques correspondantes servent de cibles d'entraîneme L'approche est dite *data-driven* car elle apprend une relation entrée-sortie au lieu de calculer uniquement une solution par optimisation sur le graphe de matching.

Résultats et interprétation. Les auteurs rapportent des performances compétitives, et dans certains régimes supérieures au matching pour le bruit au niveau du circuit sur des données simulées, alors même que le décodeur de matching dispose du modèle de bruit. Ils étudient également des données expérimentales de Google pour le code de répétition. La version révisée étend les résultats simulés à des distances allant jusqu'à $d = 9$ et ajoute une comparaison avec *belief-matching*. Ces résultats sont ceux de leur protocole, de leur architecture et de leurs données ; ils ne garantissent pas que tout GNN simple obtiendra la même performance.

Ce qu'il faut lire attentivement.

- La représentation du graphe et les caractéristiques des nœuds.
- La définition de la sortie du réseau et des classes logiques.
- Les détails du modèle de bruit et la façon de générer les données.
- La séparation entraînement/validation/test et le nombre d'échantillons.
- Les courbes de taux d'erreur logique, pas seulement l'accuracy.
- Le coût d'entraînement, le temps d'inférence et les limites d'échelle.

Rôle dans notre projet. C'est la référence méthodologique principale. Pour parler de reproduction, il faudra reprendre une configuration identifiable de l'article et comparer les résultats aux figures et paramètres correspondants. Si l'on change simultanément le graphe, l'architecture, la cible et le bruit, le projet devient une implémentation inspirée de l'article plutôt qu'une reproduction fidèle.

6.2 Higgott : PyMatching et l'appariement de poids minimal

Référence. O. Higgott, *PyMatching: A Python package for decoding quantum codes with minimum-weight perfect matching*, arXiv:2105.13082 (2021). Documentation : https://pymatching.readthedocs.io/en/stable/.

Idée mathématique du matching. Le décodeur construit un graphe dont les sommets correspondent aux détecteurs. Les mécanismes d'erreur qui provoquent un ou deux événements peuvent être représentés par des arêtes, éventuellement vers une frontière. Chaque arête porte un poids lié à la probabilité de l'événement d'erreur. Sous un modèle simple, un poids usuel est le logarithme des cotes :

$$w_e = \log \frac{1 - q_e}{q_e},$$

où q_e est la probabilité associée à l'événement représenté par l'arête. La convention précise des poids dépend de la construction du modèle. Le décodeur cherche un ensemble de chemins ou d'arêtes qui explique le syndrome avec un coût total minimal, sous les contraintes du problème d'appariement.

Le poids logarithmique se comprend par la probabilité d'un ensemble d'événements indépendants. Si E est l'ensemble des événements choisis, alors, à un facteur commun près,

$$-\log \mathbb{P}(E) = \sum_{e \in E} -\log q_e + \sum_{e \notin E} -\log(1 - q_e).$$

En retirant la somme constante des termes $-\log(1-q_e)$, on obtient un coût proportionnel à

$$\sum_{e \in E} \log \frac{1 - q_e}{q_e}.$$

Minimiser ce coût revient, dans ce modèle, à rechercher l'explication la plus probable parmi les configurations représentées. Cette interprétation a des limites : les événements peuvent être corrélés, plusieurs configurations peuvent produire le même syndrome, et le minimum de poids n'est pas toujours la décision optimale au sens de la probabilité totale d'une classe logique.

Forces et limites. PyMatching est rapide et constitue une référence robuste. Il utilise explicitement le modèle de bruit converti en DEM. Un modèle incorrect ou une représentation qui ignore certaines corrélations peut dégrader ses résultats. Inversement, il serait injuste de lui comparer un GNN en lui fournissant volontairement un modèle de bruit faux : la référence principale doit utiliser le modèle correspondant au circuit simulé.

Rôle dans notre projet. PyMatching fournit la référence quantitative principale. Le protocole doit le faire décoder exactement les mêmes syndromes que le GNN, avec la même définition d'échec logique.

6.3 Google Quantum AI : le contexte expérimental

Référence. Google Quantum AI, *Suppressing quantum errors by scaling a surface code logical qubit*, *Nature* **614**, 676-681 (2023). DOI : https://doi.org/10.1038/s41586-022-05434-1. Données : https://zenodo.org/records/6804040.

Pourquoi ce travail est important. Cette étude expérimentale montre comment la performance d'une mémoire quantique encodée dans un code de surface évolue lorsque la distance du code augmente. Elle est une référence importante pour comprendre la correction d'erreurs sur du matériel réel, où les erreurs ne suivent pas nécessairement un modèle dépolarisant indépendant et homogène.

Ce que les données ne garantissent pas. Un jeu de données expérimental ne constitue pas automatiquement un ensemble prêt à l'emploi pour entraîner n'importe quel GNN. Il faut vérifier le format des échantillons, les conventions des observables, les décodeurs déjà appliqués, la séparation entre données d'entraînement et de test, et la correspondance entre la tâche publiée et la tâche simulée. Les données peuvent permettre une extension intéressante, mais elles ne sont pas nécessaires pour valider le premier pipeline.

6.4 Mihailov et al. : une autre approche neuronale

Référence. A. Mihailov et al., *Neural network decoder for near-term surface-code experiments*, *Physical Review Research* (2025). DOI : https://doi.org/10.1103/PhysRevResearch.7.013029.

Intérêt de cette référence. Elle élargit le paysage au-delà d'un seul choix de GNN et rappelle que l'efficacité d'un décodeur neuronal dépend de son architecture, de la structure des données et du régime expérimental. Il faut lire l'article pour identifier précisément la tâche, les architectures et les conditions de comparaison avant de lui attribuer une conclusion plus spécifique.

Rôle dans notre projet. Lecture complémentaire utile pour la discussion finale : elle aide à expliquer pourquoi « réseau de neurones » n'est pas une méthode unique, et pourquoi une comparaison propre doit décrire le modèle exact.

6.5 Stim : outil de simulation et convention des détecteurs

Référence logicielle. C. Gidney, *Stim: a fast stabilizer circuit simulator*, Quantum 5, 497 (2021), DOI : https://doi.org/10.22331/q-2021-07-06-497. Code et documentation : https://github.com/quantumlib/Stim.

Stim génère des circuits bruités et échantillonne rapidement les événements de détection et les observables logiques. Les instructions de détecteur (DETECTOR) indiquent les relations de parité dont la violation signale un événement. Le DEM condense les mécanismes de bruit en relations entre événements de détection et observables. Il est crucial de comprendre que les détecteurs sont des annotations mathématiques liées au circuit, et non simplement des capteurs physiques qui identifient directement l'emplacemer de chaque erreur.

6.6 Synthèse comparative

Méthode / travail	Information exploitée	Rôle dans l'étude
PyMatching / MWPM	Syndrome + graphe d'erreurs pondéré issu du modèle de bruit	Référence algorithmique rapide
Lange et al. (GNN)	Exemples syndrome-observable et graphe annoté	Référence méthodologique à reproduire
Google et al.	Mesures d'une mémoire quantique physique et analyses de décodeurs	Contexte réel, extension optionnelle
Mihailov et al.	Modèle neuronal pour le décodage du code de surface	Alternative pour élargir l'état de l'art
Stim	Circuit bruité, détecteurs, observables et DEM	Génération des données et définition de la tâche

7 5. Que peut apporter ce projet au-delà d'une reproduction ?

7.1 Réponse honnête

Si le projet consiste uniquement à exécuter le code des auteurs sur une configuration déjà publiée et à reproduire une courbe, son apport scientifique nouveau est faible. C'est une reproduction utile pour apprendre et vérifier la compréhension, mais il ne faut pas la présenter comme une nouvelle méthode ou comme une avancée de l'état de l'art.

En revanche, une reproduction bien documentée peut devenir une étude expérimentale intéressante si elle répond à une question supplémentaire qui n'est pas déjà traitée de la même manière dans l'article. La valeur ajoutée doit être délimitée et vérifiée après lecture attentive de l'article.

7.2 Extension recommandée : généralisation au taux de bruit

L'extension la plus réaliste est de séparer deux questions :

1. **Régime interpolé** : le GNN est entraîné et testé sous le même modèle de bruit, avec des tirages indépendants.
2. **Régime décalé** : le GNN est entraîné à p_{train}, puis testé à des valeurs $p_{\text{test}} \neq p_{\text{train}}$ sans réentraînement.

Pour chaque valeur de p_{test}, PyMatching est configuré à partir du modèle de bruit correspondant au test. On mesure

$$\Delta p_{\text{L}}(p_{\text{test}}) = {p_{\text{L}}^{\text{GNN}}}_{\text{test}}(p_{\text{test}}) - {p_{\text{L}}^{\text{PM}}}_{\text{test}}(p_{\text{test}}).$$

Cette courbe indique si l'écart entre les méthodes évolue lorsque le bruit change. Il faut préciser qu'il s'agit d'un test de généralisation à un paramètre, et non d'une étude complète de robustesse à tous les défauts réalistes (bruit biaisé, dérive temporelle, erreurs corrélées, bruit de lecture non uniforme, etc.).

7.3 Extension alternative : ablation de la représentation du graphe

Si le pipeline de base fonctionne, on peut comparer deux représentations :

- **Graphe physique / DEM** : les connexions sont motivées par les relations entre détecteurs et mécanismes de faute du circuit.
- **Graphe de proximité** : les connexions sont déterminées par les k plus proches voisins des coordonnées des détecteurs.

On garde autant que possible le même modèle, la même procédure d'entraînement, les mêmes tirages et le même budget d'optimisation. On mesure si la structure d'arêtes change le taux d'erreur logique ou le temps d'inférence. Cette ablation est plus informative qu'un choix arbitraire de k, mais elle exige du temps et une définition rigoureuse des deux graphes.

7.4 Extension expérimentale Google : intérêt et limites

Des données réelles permettraient d'examiner le décalage entre une simulation idéalisée et un appareil quantique. Cependant, c'est une extension coûteuse : il faut aligner les conventions de détecteurs et d'observables, identifier les partitions de données et vérifier ce qui est réellement disponible dans l'archive. Elle ne doit être entreprise qu'après validation de bout en bout du pipeline simulé.

7.5 Quel apport peut-on revendiquer ?

Réalisation	Revendication raisonnable
Exécution du code publié sans changement	Reproduction technique et apprentissage ; pas de nouveauté méthodologique.
Réimplémentation indépendante et validation sur $d = 3, 5$	Vérification reproductible d'un protocole sur un périmètre explicite.
Comparaison contrôlée entraînement/test à taux identique et différent	Étude de sensibilité au décalage de taux de bruit, si elle n'est pas déjà couverte de façon équivalente par la référence choisie.
Ablation graphe DEM contre proximité	Analyse de l'influence de la représentation, si le protocole est suffisamment contrôlé.
Test sur données expérimentales correctement alignées	Évaluation complémentaire sur des données réelles ; conclusion limitée au jeu de données disponible.

La formulation finale de l'apport doit dépendre de ce que révèle la lecture de l'article. Si l'étude de généralisation ou l'ablation est déjà faite par Lange et al. dans des conditions comparables, il faudra soit reproduire explicitement cette partie, soit trouver une différence précise et défendable dans le protocole. Il ne faut pas revendiquer une nouveauté simplement parce qu'une expérience est nouvelle pour l'étudiant.

8 6. Planning opérationnel de 20 heures

Tâche	**Durée**
Lecture ciblée de Lange, Stim et PyMatching ; définition du protocole	1 h
Pipeline Stim $\rightarrow$ PyMatching et validation du taux logique	2 h
Référence statistique, décodeur trivial et préparation du test	2 h
Construction et vérification des graphes	3 h
Implémentation, entraînement et validation du GNN	5 h
Comparaison des taux logiques et mesures de temps	3 h
Test à un taux de bruit différent, si le pipeline est stable	2 h
Rédaction, figures et limites	2 h
Total	**20 h**

8.1 Critères de réussite

Le projet est réussi si :

1. le circuit et les paramètres de bruit sont documentés ;
2. le calcul de l'échec logique est validé sur un exemple contrôlé ;
3. PyMatching est évalué sur un jeu de test clairement défini ;

4. le GNN apprend sur les données d'entraînement et est évalué sur un test indépendant ;
5. les taux d'erreur sont accompagnés du nombre de tirs et d'une incertitude statistique ;
6. le résultat négatif éventuel est analysé plutôt que masqué ;
7. le code, les graines, les versions de bibliothèques et les paramètres sont consignés.

8.2 Risques et réponses prévues

Risque	Conséquence	Réponse
Taux d'échec très faible	Estimation statistique imprécise	Augmenter les tirs, commencer par un pilote et rapporter une borne plutôt que conclure à zéro.
GNN n'apprend pas	Pas de comparaison significative	Vérifier la cible, le déséquilibre des classes, la représentation du graphe et la fuite de données.
Graphe vide ou mal construit	Exemples perdus ou représentations incohérentes	Comparer graphe de détecteurs activés et graphe avec tous les détecteurs potentiels.
Temps d'entraînement trop long	Dépassement du budget	Réduire la taille du jeu, la dimension cachée ou le nombre de couches pour le prototype ; ne pas changer le test final.
Résultat différent de l'article	Impossible de conclure directement à un échec de reproduction	Vérifier le circuit, les paramètres de bruit, les cibles, les splits, l'architecture et les hyperparamètres.
Données expérimentales difficiles à aligner	Extension trop coûteuse	Garder l'extension Google hors du périmètre principal.

9 7. Reproductibilité et bonnes pratiques

Le dépôt du projet devrait contenir :

- un fichier de dépendances avec les versions de Python, Stim, PyMatching, PyTorch et PyTorch Geometric ;
- un script de génération des circuits et des données ;
- un script de construction du modèle PyMatching ;
- un script d'entraînement du GNN ;

- un script d'évaluation commun aux deux décodeurs ;
- un fichier de configuration listant d, les rounds, tous les taux de bruit, les graines, les tailles des ensembles et les hyperparamètres ;
- les résultats bruts (nombre de tirs, nombre d'échecs, durées) et les scripts qui produisent les figures.

Ne pas conserver uniquement les courbes finales : enregistrer également les comptes d'échecs et le nombre de tirs permet de recalculer les taux et les intervalles de confiance. Les résultats doivent pouvoir être régénérés à partir des paramètres déclarés.

10 8. Références bibliographiques

Références

[1] M. Lange, P. Havström, B. Srivastava, I. Bengtsson, V. Bergentall, K. Hammar, O. Heuts, E. van Nieuwenburg et M. Granath, *Data-driven decoding of quantum error correcting codes using graph neural networks*, Physical Review Research 7, 023181 (2025). DOI : https://doi.org/10.1103/PhysRevResearch.7.023181. Prépublication : https://arxiv.org/abs/2307.01241.

[2] O. Higgott, *PyMatching: A Python package for decoding quantum codes with minimum-weight perfect matching*, arXiv:2105.13082 (2021). https://arxiv.org/abs/2105.13082.

[3] C. Gidney, *Stim: a fast stabilizer circuit simulator*, Quantum 5, 497 (2021). DOI : https://doi.org/10.22331/q-2021-07-06-497. Code : https://github.com/quantumlib/Stim.

[4] Google Quantum AI, *Suppressing quantum errors by scaling a surface code logical qubit*, Nature 614, 676-681 (2023). DOI : https://doi.org/10.1038/s41586-022-05434-1. Données : https://zenodo.org/records/6804040.

[5] A. Mihailov et al., *Neural network decoder for near-term surface-code experiments*, Physical Review Research (2025). DOI : https://doi.org/10.1103/PhysRevResearch.7.013029.

[6] Quantumlib, *Stim documentation: detector error model and sampling*. https://github.com/quantumlib/Stim.

[7] O. Higgott et al., *PyMatching documentation*. https://pymatching.readthedocs.io/en/stable/.

[8] PyTorch Geometric contributors, *PyTorch Geometric documentation*. https://pytorch-geometric.readthedocs.io/.